

## Chapitre IV : Les Types Composés

- **Author** : M. ATTOISSE Mohamed
- **School** : *Institut Supérieur d'Informatique*
- **Level** : *Licence I*
- **Period** : *2024-2025*
- **Subject** : *Langage C*



## Problematique

Imaginons un programme qui doit gérer des informations sur des étudiants dans une université. Chaque étudiant a plusieurs attributs :

- Un nom (`char`) , un âge (`int`) et une moyenne (`float`) .

Sans types composés, nous serions obligés de manipuler plusieurs variables séparées pour chaque étudiant, ce qui rendrait le code :

- Peu lisible : Difficile de comprendre quelles variables sont liées.
- Peu maintenable : Toute modification nécessiterait des changements dans plusieurs parties du code.
- Peu efficace : La gestion des données deviendrait fastidieuse et sujette à des erreurs.

## Solution

Pour résoudre ce problème, nous devons utiliser les types composés.

## Introduction

Les types composés offrent une méthode puissante pour organiser et manipuler des données de manière structurée. Ils permettent de regrouper plusieurs valeurs de différents types sous un seul nom, améliorant ainsi la lisibilité, la maintenabilité et l'efficacité du code. On distingue 3 types composés :

- **Enumération**
- **Union**
- **Enregistrement**

## Énumération `enum`

Les énumérations en C permettent de définir un ensemble de constantes nommées associées à des valeurs entières. Elles sont utiles pour rendre le code plus lisible en remplaçant les valeurs numériques arbitraires par des symboles significatifs.

### Syntaxe

```
enum NomDeLEnumeration {  
    constante1,  
    constante2,  
    // ...  
};
```

## Union union

Les unions permettent de stocker différents types de données dans le même espace mémoire. Cependant, seule une valeur peut être stockée à la fois dans une union.

### Syntaxe

```
union NomDeLUnion {  
    type1 membre1;  
    type2 membre2;  
    // ...  
};
```

Dans ce cours, nous allons nous concentrer sur un type composé spécifique : **la structure**, également appelée enregistrement en français ou struct en anglais.

## Enregistrement `struct`

La structure est un outil puissant pour regrouper plusieurs variables de types différents sous un même nom, ce qui permet d'organiser et de manipuler des données de manière structurée.

### Syntaxe 1

```
struct NomDeLaStructure {  
    type1 membre1;  
    type2 membre2;  
    // ...  
};  
  
struct NomDeLaStructure NomVariable;
```

**Exemple 1 :** créer un type Point.

```
struct Point {  
    float x;  
    float y;  
};  
struct Point P;
```

## Syntaxe 2

```
struct NomDeLaStructure {  
    type1 membre1;  
    type2 membre2;  
    // ...  
}NomVariable;
```

**Exemple 2 :** Définir la structure d'un étudiant. ce dernier est caractérisé par son nom, son âge et sa moyenne.

```
struct Etudiant {  
    char nom[20];  
    int age;  
    float moyenne;  
}E;
```

## Utilisation de **TYPEDEF** typedef

**TYPEDEF** est un mot-clé utilisé pour créer des alias de types existants. Cela permet de définir des noms significatifs pour les types de données, améliorant ainsi la lisibilité du code.

### Syntaxe

```
typedef AncienType NouveauType;
```

**Exemple :** Renommer un `int` en entier

```
typedef int entier;
```

**NB :** *Pour accéder aux champs d'une variables de type structure S, il faut faire suivre du Nom un point(.) puis le nom du champ. (nomVariableEnregistrement.champ)*

**Exemple :** Pour accéder au champ `age` de type `Etudiant` il faut faire **E.age**

**Application :** Ecrire un programme qui permet de saisir les données de N produits. Chaque produit est caractérisé par sa désignation, prix et quantité . Le programme affiche les données de chaque produit puis détermine et affiche le montant total.